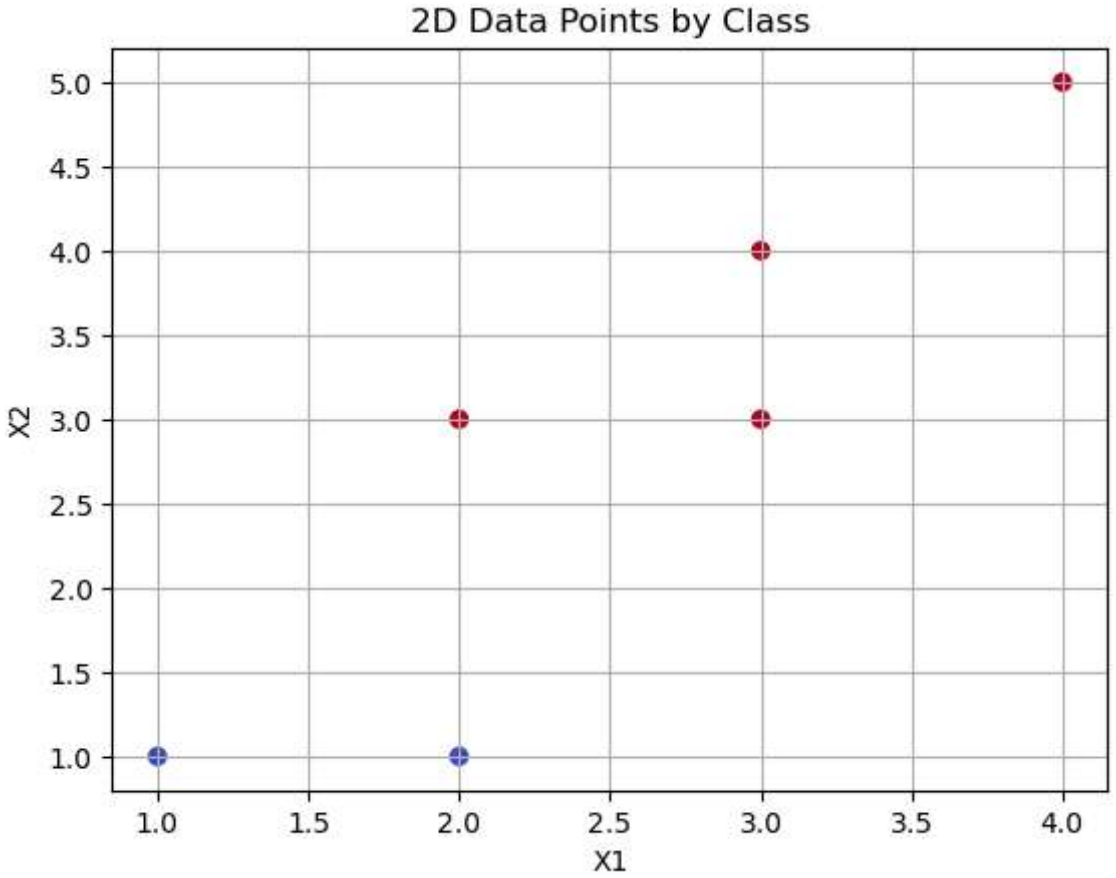
PART A: Prerequisite for Decision Tree Implementation

1. Create a small dataset using numpy

In [1]:
```
import numpy as np

x=[[1,1],[2,1],[2,3],[3,3],[3,4],[4,5]]
y=[0,0,1,1,1,1]

x=np.array(x)
y=np.array(y)
```

2. Plot data points in 2D with class colors

In [2]:
```
import matplotlib.pyplot as plt

plt.scatter(x[:,0], x[:,1], c=y, cmap='coolwarm')
plt.title("2D Data Points by Class")
plt.xlabel("X1")
plt.ylabel("X2")
plt.grid(True)
plt.show()
```



3. Compute entropy

In [3]:
```
from math import log2

def compute_entropy(y):
    classes, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)
```

```
    entropy = -np.sum(probabilities * np.log2(probabilities))
    return entropy

print(compute_entropy(y))
```

```
0.9182958340544896
```

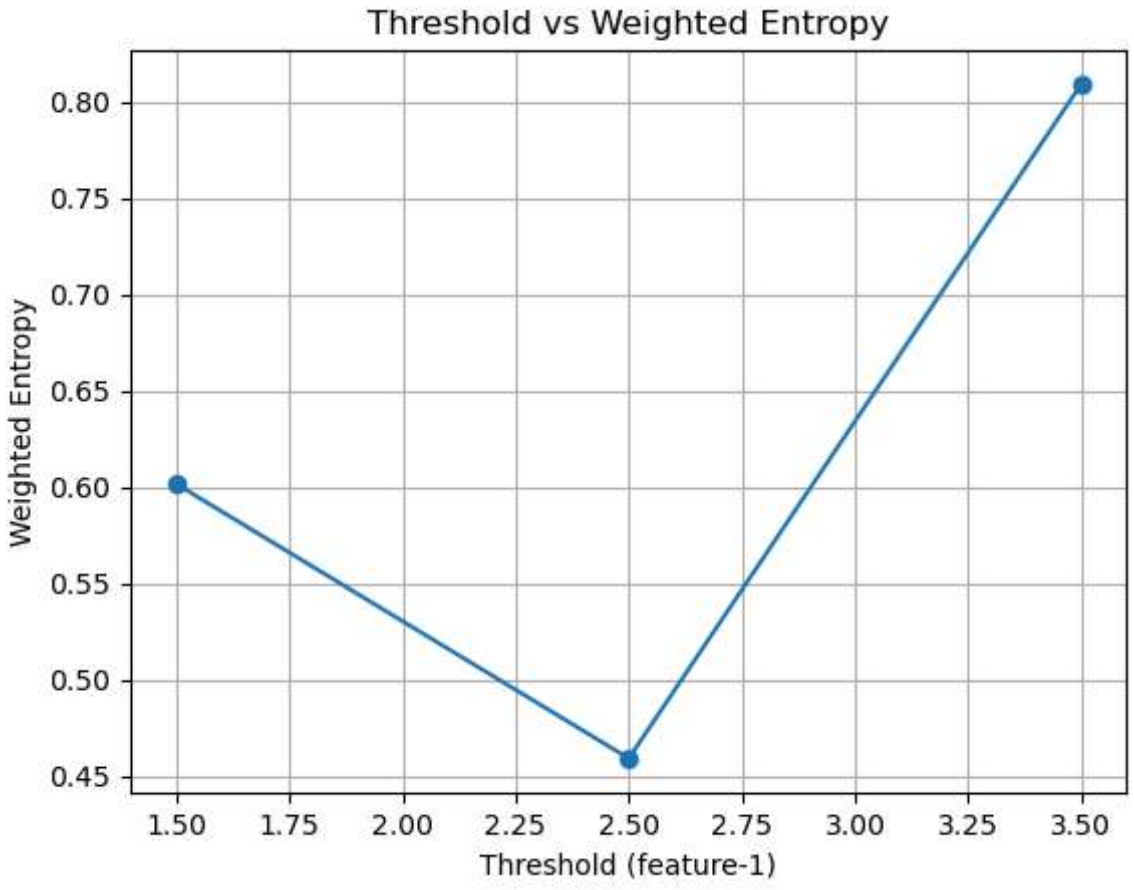
4. Compute weighted entropy for feature-1 thresholds [1.5,2.5,3.5]

In [4]:
```
thresholds = [1.5, 2.5, 3.5]
weighted_entropy = []

for t in thresholds:
    left = y[x[:,0] <= t]
    right = y[x[:,0] > t]
    we = (len(left)/len(y))*compute_entropy(left) + (len(right)/len(y))*compute_ent
    weighted_entropy.append(we)

print(weighted_entropy)
```

```
[0.6016067457394686, 0.4591479170272448, 0.8091254953788906]
```

5. Plot threshold vs weighted entropy

In [5]:
```
plt.plot(thresholds, weighted_entropy, marker='o')
plt.xlabel("Threshold (feature-1)")
plt.ylabel("Weighted Entropy")
plt.title("Threshold vs Weighted Entropy")
plt.grid(True)
plt.show()
```



PART B: Decision Tree using Scikit-Learn

1. Load the CSV dataset (drug200.csv)

In [6]:
```
import pandas as pd

df=pd.read_csv('drug200.csv')
df.head(5)
```

Out[6]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY

2. Perform label encoding in the categorical columns.

In [7]:
```
from sklearn.preprocessing import LabelEncoder

le=LabelEncoder()
df['Sex']=le.fit_transform(df['Sex'])
df['BP']=le.fit_transform(df['BP'])
df['Cholesterol']=le.fit_transform(df['Cholesterol'])
```

In [8]:
```
df.head()
```

Out[8]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	0	0	0	25.355	drugY
1	47	1	1	0	13.093	drugC
2	47	1	1	0	10.114	drugC
3	28	0	2	0	7.798	drugX
4	61	0	1	0	18.043	drugY

3. Define features and target, and perform train-test split

In [9]:
```
from sklearn.model_selection import train_test_split

x=df.drop('Drug',axis=1)
y=df['Drug']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

4. Train a Decision Tree classifier with entropy

In [10]:
```
from sklearn.tree import DecisionTreeClassifier

dtc=DecisionTreeClassifier(max_depth=3,criterion="entropy")

dtc.fit(x_train,y_train)
```

Out[10]:

DecisionTreeClassifier

DecisionTreeClassifier(criterion='entropy', max_depth=3)

In [11]:
```
y_pred=dtc.predict(x_test)
```

5. Compute accuracy, precision, recall, f1-score

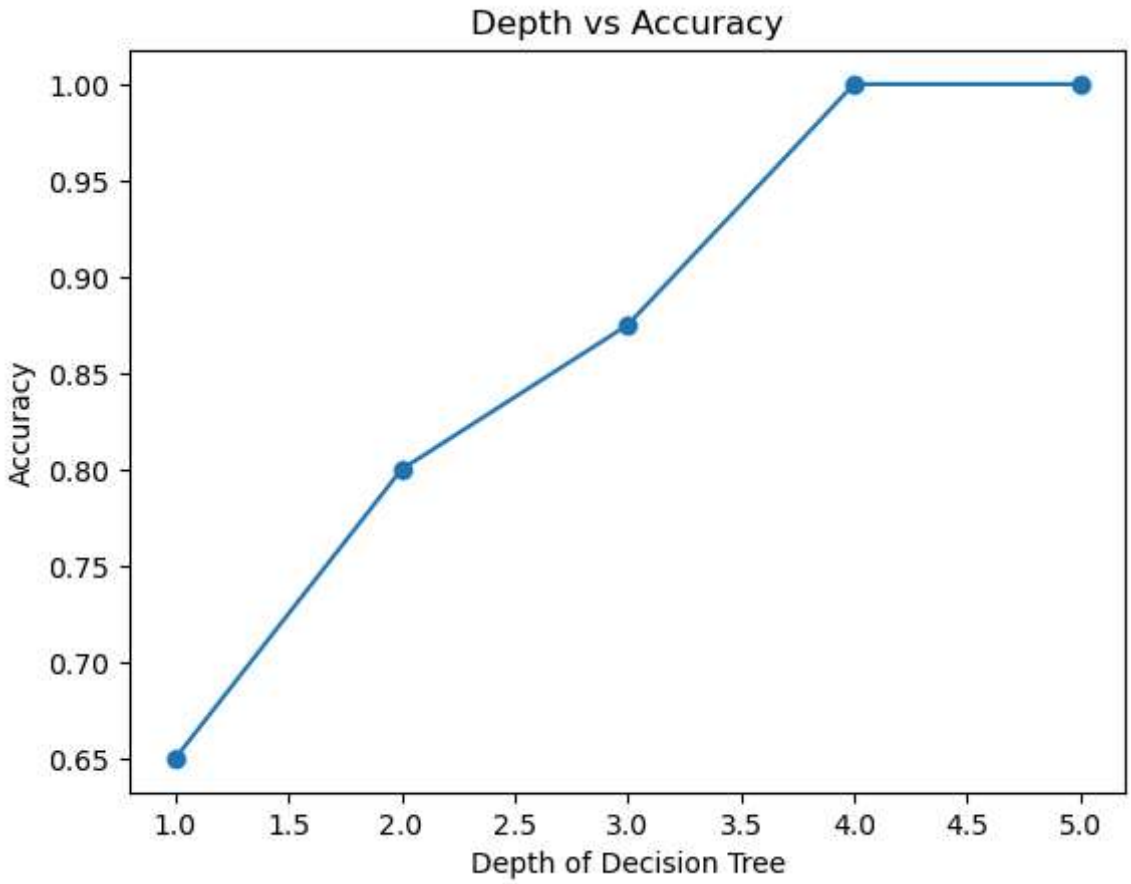
In [12]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

acc = accuracy_score(y_test, y_pred)
pre = precision_score(y_test, y_pred, average='macro', zero_division=0)
rec = recall_score(y_test, y_pred, average='macro', zero_division=0)
f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)


print("Accuracy:", acc)
print("Precision:", pre)
print("Recall:", rec)
print("F1-score:", f1)
```

```
Accuracy: 0.875
Precision: 0.7375
Recall: 0.8
F1-score: 0.762962962962963
```

PART C

1. Change depth [1,2,3,4,5] and plot depth vs accuracy

In [13]:
```
depths=[1,2,3,4,5]
scores=[]

for d in depths:
    dtc=DecisionTreeClassifier(max_depth=d,criterion="entropy")
    dtc.fit(x_train,y_train)
    y_pred=dtc.predict(x_test)
    acc_d=accuracy_score(y_test,y_pred)
    scores.append(acc_d)

plt.plot(depths,scores,marker='o')
plt.xlabel("Depth of Decision Tree")
plt.ylabel("Accuracy")
plt.title("Depth vs Accuracy")
plt.show()
```



In [14]:
```
dtc_gini=DecisionTreeClassifier(max_depth=3,criterion="gini")
dtc_gini.fit(x_train,y_train)
y_pred_gini=dtc_gini.predict(x_test)

acc_gini=accuracy_score(y_test,y_pred_gini)

print("Accuracy with Entropy: ",acc)
print("Accuracy with Gini: ",acc_gini)
```

```
Accuracy with Entropy:  0.875
Accuracy with Gini:  0.875
```